

colourPalette.js

//Displays and handles the colour palette.

```
function ColourPalette() {  
  //a list of web colour strings  
  this.colours = ["black", "silver", "gray", "white", "maroon",  
"red", "purple",  
    "orange", "pink", "fuchsia", "green", "lime", "olive",  
"yellow", "navy",  
    "blue", "teal", "aqua"  
];  
  //make the start colour be black  
  this.selectedColour = "black";  
  
  var self = this;  
  
  var colourClick = function() {  
    //remove the old border  
    var current = select("#" + self.selectedColour + "Swatch");  
    current.style("border", "0");  
  
    //get the new colour from the id of the clicked element  
    var c = this.id().split("Swatch")[0];  
  
    //set the selected colour and fill and stroke  
    self.selectedColour = c;  
    fill(c);  
    stroke(c);  
  
    //add a new border to the selected colour  
    this.style("border", "2px solid blue");  
  }  
  
  //load in the colours  
  this.loadColours = function() {  
    //set the fill and stroke properties to be black at the start  
of the programme  
    //running  
    fill(this.colours[0]);  
    stroke(this.colours[0]);  
  
    //for each colour create a new div in the html for the  
colourSwatches  
    for (var i = 0; i < this.colours.length; i++) {  
      var colourID = this.colours[i] + "Swatch";  
  
      //using p5.dom add the swatch to the palette and set its  
background colour  
      //to be the colour value.
```

```

        var colourSwatch = createDiv()
        colourSwatch.class('colourSwatches');
        colourSwatch.id(colourID);

        select(".colourPalette").child(colourSwatch);
        select("#" + colourID).style("background-color",
this.colours[i]);
        colourSwatch.mouseClicked(colourClick)
    }

    select(".colourSwatches").style("border", "2px solid blue");
};
//call the loadColours function now it is declared
this.loadColours();
}

```

ellipseTool.js

//TODO - Edge of ellipse still aren't exactly where cursor is.
//TODO - Circle follows cursor around the canvas.

```

function ellipseTool() {
    this.icon = "assets/ellipse.png";
    this.name = "Ellipse";
    this.description = "Ellipse: Hold Shift for circle";

    var startMouseX = -1;
    var startMouseY = -1;
    var drawing = false;

    this.draw = function () {
        strokeWeight(setStrokeWeight);
        if (mouseIsPressed) {
            if (startMouseX == -1) {
                startMouseX = mouseX;
                startMouseY = mouseY;
                drawing = true;
                loadPixels();
            } else {
                updatePixels();
                applyShapeFill();

                // Variables to enable drawing from corner instead
                // of center, as that seems more natural for painting app

                let w = mouseX - startMouseX;
                let h = mouseY - startMouseY;
                // With shift pressed, draw a circle
                if (keyIsDown(SHIFT)) {
                    ellipse(

```

```

        startMouseX + w / 2,
        startMouseY + h / 2,
        // Take the smallest (absolute) of two values
        min(abs(w), abs(h)),
    );
} else {
    // Otherwise draw a regular/irregular ellipse
    ellipse(startMouseX + w / 2, startMouseY + h / 2, w, h);
}
}
} else if (drawing) {
    drawing = false;
    startMouseX = -1;
    startMouseY = -1;
}
};
}

```

freehandTool.js

```

function FreehandTool(){
    //set an icon and a name for the object
    this.icon = "assets/freehand.jpg";
    this.name = "freehand";
    this.description = "Freehand drawing tool"

    //to smoothly draw we'll draw a line from the previous mouse
    location
    //to the current mouse location. The following values store
    //the locations from the last frame. They are -1 to start with
    because
    //we haven't started drawing yet.
    var previousMouseX = -1;
    var previousMouseY = -1;

    this.draw = function(){
        strokeWeight(setStrokeWeight);
        //if the mouse is pressed
        if(mouseIsPressed){
            //check if they previousX and Y are -1. set them to the
            current
            //mouse X and Y if they are.
            if (previousMouseX == -1){
                previousMouseX = mouseX;
                previousMouseY = mouseY;
            }
            //if we already have values for previousX and Y we can
            draw a line from
            //there to the current mouse location
            else{
                line(previousMouseX, previousMouseY, mouseX, mouseY);
            }
        }
    }
}

```

```

        previousMouseX = mouseX;
        previousMouseY = mouseY;
    }
}
//if the user has released the mouse we want to set the
previousMouse values
//back to -1.
//try and comment out these lines and see what happens!
else{
    previousMouseX = -1;
    previousMouseY = -1;
}
};
}

```

helperFunctions.js

```

function HelperFunctions() {
    //p5.dom click click events. Notice that there is no this. at the
    //start we don't need to do that here because the event will
    //be added to the button and doesn't 'belong' to the object

    //event handler for the clear button event. Clears the screen
    select("#clearButton").mouseClicked(function () {
        // Clear button simply sets background to white, effectively
        wiping all
        // exisiting drawing off
        background(255);

        //call loadPixels to update the drawing state
        //this is needed for the mirror tool
        loadPixels();
    });

    //event handler for the save image button. saves the canvsa to the
    //local file system.
    select("#saveImageButton").mouseClicked(function () {
        // Using built in saveCanvas funtion, only two parameters needed
        are filename and extension,
        saveCanvas("Drawing", "jpg");
    });

    // START OWN CODE
    // Toggles fill button between filled and non filled shape
    select("#fillButton").mouseClicked(function () {
        fillShapes = !fillShapes;

        if (fillShapes) {
            this.html("Solid");
        } else {
            this.html("Outline");
        }
    });
}

```

```

    }
  });

  var alphaSlider = select("#alphaSlider");
  var alphaValue = select("#alphaValue");

  alphaSlider.input(function () {
    fillAlpha = Number(alphaSlider.value());

    // Update the display number shown beside the slider
    alphaValue.html(fillAlpha);
  });

  // Display values
  var strokeWeightSlider = select("#strokeWeight");
  var strokeWeightValue = select("#strokeValue");
  //Get value from slider and convert to number
  strokeWeightSlider.input(function () {
    setStrokeWeight = Number(strokeWeightSlider.value());

    // Update the display number beside the slider
    strokeWeightValue.html(setStrokeWeight);
  });
}

// Carry out the filling of the shape in color fetched from colour
// palette
function applyShapeFill() {
  if (!fillShapes) {
    noFill();
    return;
  }

  fill(
    red(colourP.selectedColour),
    green(colourP.selectedColour),
    blue(colourP.selectedColour),
    fillAlpha,
  );
}
// END OWN CODE

```

lineToTool.js

```

function LineToTool() {
  // Set icon for tool to be displayed in the toolbox
  this.icon = "assets/lineTo.jpg";
  // Tool name so it can be referred to
  this.name = "LineTo";
}

```

```

    this.description = "Straight Line Tool"

    // Stores coordinats for where to start drawing from, -1
    placeholders simply fork as flags
    // to indicate that no drawing is being made yet
    var startMouseX = -1;
    var startMouseY = -1;
    var drawing = false;

    // Function is being called continuously, while tool is selected
    this.draw = function () {
        strokeWeight(setStrokeWeight)
        // If tool selected, and mouse pressed, start drawing
        if (mouseIsPressed) {
            // If drawing is not started yet, set coordinates where mouse
            button was pressed as start coordinates
            if (startMouseX == -1) {
                startMouseX = mouseX;
                startMouseY = mouseY;
                // Flag drawing as in progress
                drawing = true;
                // Needed so only the single active line is stored not full
                trailing history
                loadPixels();
            } else {
                // Restores the saved canvas
                updatePixels();
                // Draws line from coordinates whee mouse button was pressed,
                to where it is now
                line(startMouseX, startMouseY, mouseX, mouseY);
            }
        }
        // Kicks in when mouse button is released
        else if (drawing) {
            // Resets tool parameters to initial ones.
            drawing = false;
            startMouseX = -1;
            startMouseY = -1;
        }
    };
}

```

mirrorDrawTool.js

```

function mirrorDrawTool() {
    this.name = "mirrorDraw";
    this.icon = "assets/mirrorDraw.jpg";
    this.description = "Mirror drawing tool"

    //which axis is being mirrored (x or y) x is default

```

```

this.axis = "x";
//line of symmetry is halfway across the screen
this.lineOfSymmetry = width / 2;

//this changes in the p5.dom click handler. So storing it as
//a variable self now means we can still access this in the
handler
var self = this;

//where was the mouse on the last time draw was called.
//set it to -1 to begin with
var previousMouseX = -1;
var previousMouseY = -1;

//mouse coordinates for the other side of the Line of symmetry.
var previousOppositeMouseX = -1;
var previousOppositeMouseY = -1;

this.draw = function() {
  //display the last save state of pixels
  updatePixels();

  //do the drawing if the mouse is pressed
  if (mouseIsPressed) {
    //if the previous values are -1 set them to the current
    mouse location
    //and mirrored positions
    if (previousMouseX == -1) {
      previousMouseX = mouseX;
      previousMouseY = mouseY;
      previousOppositeMouseX =
this.calculateOpposite(mouseX, "x");
      previousOppositeMouseY =
this.calculateOpposite(mouseY, "y");
    }

    //if there are values in the previous locations
    //draw a line between them and the current positions
    else {
      line(previousMouseX, previousMouseY, mouseX, mouseY);
      previousMouseX = mouseX;
      previousMouseY = mouseY;

      //these are for the mirrored drawing the other side of
the
      //line of symmetry
      var oX = this.calculateOpposite(mouseX, "x");
      var oY = this.calculateOpposite(mouseY, "y");
      line(previousOppositeMouseX, previousOppositeMouseY,

```

```

oX, oY);
        previousOppositeMouseX = oX;
        previousOppositeMouseY = oY;
    }
}
//if the mouse isn't pressed reset the previous values to -1
else {
    previousMouseX = -1;
    previousMouseY = -1;

    previousOppositeMouseX = -1;
    previousOppositeMouseY = -1;
}

//after the drawing is done save the pixel state. We don't
want the
//line of symmetry to be part of our drawing

loadPixels();

//push the drawing state so that we can set the stroke weight
and colour
push();
strokeWeight(3);
stroke("red");
//draw the line of symmetry
if (this.axis == "x") {
    line(width / 2, 0, width / 2, height);
} else {
    line(0, height / 2, width, height / 2);
}
//return to the original stroke
pop();

};

/*calculate an opposite coordinate the other side of the
*symmetry line.
*@param n number: location for either x or y coordinate
*@param a [x,y]: the axis of the coordinate (x or y)
*@return number: the opposite coordinate
*/
this.calculateOpposite = function(n, a) {
    //if the axis isn't the one being mirrored return the same
    //value
    if (a != this.axis) {
        return n;
    }
}

```

```

        //if n is less than the line of symmetry return a coordinate
        //that is far greater than the line of symmetry by the
distance from
        //n to that line.
        if (n < this.lineOfSymmetry) {
            return this.lineOfSymmetry + (this.lineOfSymmetry - n);
        }

        //otherwise a coordinate that is smaller than the line of
symmetry
        //by the distance between it and n.
        else {
            return this.lineOfSymmetry - (n - this.lineOfSymmetry);
        }
    };

    //when the tool is deselected update the pixels to just show the
drawing and
    //hide the line of symmetry. Also clear options
    this.unselectTool = function() {
        updatePixels();
        //clear options
        select(".options").html("");
    };

    //adds a button and click handler to the options area. When
clicked
    //toggle the line of symmetry between horizontal to vertical
    this.populateOptions = function() {
        select(".options").html(
            "<button id='directionButton'>Make Horizontal</button>");
        // //click handler
        select("#directionButton").mouseClicked(function() {
            var button = select("#" + this.elt.id);
            if (self.axis == "x") {
                self.axis = "y";
                self.lineOfSymmetry = height / 2;
                button.html('Make Vertical');
            } else {
                self.axis = "x";
                self.lineOfSymmetry = width / 2;
                button.html('Make Horizontal');
            }
        });
    };
}

```

polygonTool.js

```
function polygonTool() {  
  // This one took a while to get working, but was wrth it in the end.  
  // Getting both preview and then the finished vertexes behaving as  
  they should  
  // was trickier that i anticipated  
  this.icon = "assets/polygon.jpg";  
  this.name = "Polygon Tool";  
  
  //Store the confirmed vertices  
  let clickedPoints = [];  
  let drawing = false;  
  // Track mouse button sgtate to detect start of the polygon  
  let previousMouse = false;  
  
  this.draw = function () {  
    strokeWeight(setStrokeWeight);  
    // Detect a new mouse click inside drawing area  
    // Check for click, state, and whether the click was on actual  
    canvas  
    // Initially it did take clicks on buttons and icons as a point  
    for vertex.  
    if (  
      mouseIsPressed &&  
      !previousMouse &&  
      mouseX >= 0 &&  
      mouseX <= width &&  
      mouseY >= 0 &&  
      mouseY <= height  
    ) {  
      if (!drawing) {  
        drawing = true;  
        // Clear the vertices array  
        clickedPoints = [];  
        loadPixels();  
      }  
      // Push the clicked point into vertices array  
      clickedPoints.push({  
        x: mouseX,  
        y: mouseY,  
      });  
    }  
  
    // Set previous mouse to true if shape have been started  
    previousMouse = mouseIsPressed;  
  
    if (!drawing) {  
      return;  
    }  
  }  
}
```

```

updatePixels();
// Already have it, why not use it, or atleast give user option to
applyShapeFill();

beginShape();
// Loop through the array and draw all vertices
for (let i = 0; i < clickedPoints.length; i++) {
  vertex(clickedPoints[i].x, clickedPoints[i].y);
}

// Preview of the line currently being drawn
vertex(mouseX, mouseY);

endShape();

// Press Enter to finish drawing, and connect last vertice back to
the start point
// to create finished shape
if (keyIsDown(ENTER) && clickedPoints.length >= 3) {
  updatePixels();
  applyShapeFill();

  beginShape();

  for (let i = 0; i < clickedPoints.length; i++) {
    vertex(clickedPoints[i].x, clickedPoints[i].y);
  }

  endShape(CLOSE);

  loadPixels();

  drawing = false;
  clickedPoints = [];
}
};
}

```

rectangleTool.js

```

function rectangleTool() {
  this.icon = "assets/rectangle.png";
  this.name = "Rectangle";
  this.description = "Rectangle tool. Hold Shift for square"

  // Will start it the same as lineToTool, as they are in a way
  similar
  var startMouseX = -1;
  var startMouseY = -1;

```

```

var drawing = false;

this.draw = function () {
  // noFill() is left only for use with rect tool
  //noFill();
  strokeWeight(setStrokeWeight)
  if (mouseIsPressed) {
    if (startMouseX == -1) {
      startMouseX = mouseX;
      startMouseY = mouseY;
      drawing = true;

      loadPixels();
    } else {
      updatePixels();
      applyShapeFill();
      //// This would be the easy/sensible aproach
      // rect(
      //           startX,
      //           startY,
      //           mouseX - startX,
      //           mouseY - startY
      // )

      // // But for the sake of understanding, lets use lines
      // // Top
      // line(startMouseX, startY, mouseX, startY);
      // // Right side
      // line(mouseX, startY, mouseX, mouseY);
      // // Bottom
      // line(startMouseX, mouseY, mouseX, mouseY);
      // // Left side
      // line(startMouseX, startY, startX, mouseY);

      // Had to switch to vertexes so i can have a properly closed
      shape for solid/outline shapes
      beginShape();
      vertex(startMouseX, startY);
      vertex(mouseX, startY);
      vertex(mouseX, mouseY);
      vertex(startMouseX, mouseY);
      endShape(CLOSE);
    }
  } else if (drawing) {
    // Reset parameters
    drawing = false;
    startMouseX = -1;
    startMouseY = -1;
  }
}

```

```
};  
}
```

sketch.js

```
//global variables that will store the toolbox colour palette  
//amnd the helper functions  
var toolbox = null;  
var colourP = null;  
var helpers = null;  
  
// START OWN CODE  
  
// Global flag for the outline/solid helper tool  
var fillShapes = false;  
// Alpha variable for transparency slider, default at completely solid  
var fillAlpha = 255;  
// Variable for line thickness adjustment, default at 1  
var setStrokeWeight = 1;  
// END OWN CODE  
  
function setup() {  
  //create a canvas to fill the content div from index.html  
  canvasContainer = select("#content");  
  var c = createCanvas(  
    canvasContainer.size().width,  
    canvasContainer.size().height,  
  );  
  c.parent("content");  
  
  //create helper functions and the colour palette  
  helpers = new HelperFunctions();  
  colourP = new ColourPalette();  
  
  //create a toolbox for storing the tools  
  toolbox = new Toolbox();  
  
  //add the tools to the toolbox.  
  toolbox.addTool(new FreehandTool());  
  toolbox.addTool(new LineToTool());  
  toolbox.addTool(new mirrorDrawTool());  
  // MY OWN CODE  
  toolbox.addTool(new rectangleTool());  
  toolbox.addTool(new ellipseTool());  
  toolbox.addTool(new polygonTool());  
  
  // MY OWN CODE END  
  toolbox.addTool(new sprayCanTool());  
  background(255);  
}
```

```

function draw() {
    //call the draw function from the selected tool.
    //hasOwnProperty is a javascript function that tests
    //if an object contains a particular method or property
    //if there isn't a draw method the app will alert the user
    if (toolbox.selectedTool.hasOwnProperty("draw")) {
        toolbox.selectedTool.draw();
    } else {
        alert("it doesn't look like your tool has a draw method!");
    }
}

```

sprayCanTool.js

```

function sprayCanTool() {
    this.name = "sprayCanTool";
    this.icon = "assets/sprayCan.jpg";
    this.points = 13;
    this.spread = 10;
    this.draw = function () {
        //if the mouse is pressed paint on the canvas
        //spread describes how far to spread the paint from the mouse
        //pointer
        //points holds how many pixels of paint for each mouse press.
        if (mouseIsPressed) {
            for (var i = 0; i < this.points; i++) {
                point(
                    random(mouseX - this.spread, mouseX + this.spread),
                    random(mouseY - this.spread, mouseY + this.spread),
                );
            }
        }
    };
}

```

toolbox.js

//container object for storing the tools. Functions to add new tools and select a tool

```

function Toolbox() {
    var self = this;

    this.tools = [];
    this.selectedTool = null;

    var toolbarItemClick = function () {
        //remove any existing borders
        var items = selectAll(".sideBarItem");
        for (var i = 0; i < items.length; i++) {
            items[i].style("border", "0");
        }
    }
}

```

```

var toolName = this.id().split("sideBarItem")[0];
self.selectTool(toolName);

//call loadPixels to make sure most recent changes are saved to
pixel array
loadPixels();
};

//add a new tool icon to the html page
var addToolIcon = function (icon, name, description) {
    var sideBarItem = createDiv("<img src='" + icon + "'></div>");
    sideBarItem.class("sideBarItem");
    sideBarItem.id(name + "sideBarItem");
    sideBarItem.attribute("title", description);
    sideBarItem.parent("sidebar");
    sideBarItem.mouseClicked(toolbarItemClick);
};

//add a tool to the tools array
this.addTool = function (tool) {
    //check that the object tool has an icon and a name
    if (!tool.hasOwnProperty("icon") || !tool.hasOwnProperty("name"))
    {
        alert("make sure your tool has both a name and an icon");
    }
    this.tools.push(tool);
    // MY OWN CODE START
    addToolIcon(tool.icon, tool.name, tool.description);
    // MY OWN CODE END
    //if no tool is selected (ie. none have been added so far)
    //make this tool the selected one.
    if (this.selectedTool == null) {
        this.selectTool(tool.name);
    }
};

this.selectTool = function (toolName) {
    //search through the tools for one that's name matches
    //toolName
    for (var i = 0; i < this.tools.length; i++) {
        if (this.tools[i].name == toolName) {
            //if the tool has an unselectTool method run it.
            if (
                this.selectedTool != null &&
                this.selectedTool.hasOwnProperty("unselectTool")
            ) {
                this.selectedTool.unselectTool();
            }
            //select the tool and highlight it on the toolbar

```

```
    this.selectedTool = this.tools[i];
    select("#" + toolName + "sideBarItem").style(
        "border",
        "2px solid blue",
    );

    //if the tool has an options area. Populate it now.
    if (this.selectedTool.hasOwnProperty("populateOptions")) {
        this.selectedTool.populateOptions();
    }
}
};
}
```